IIA • WEB DESIGN

Introduction to JavaScript

UNIT 10 | SIMPLE INTERACTIONS • BUTTONS • DOM • EVENTS

Library and Information Studies • Records Management Systems

Learning outcomes

Understand the role of JavaScript

HTML gives content meaning; CSS controls its presentation; JavaScript can add meaningful behaviour when visitors interact with the page.

Practise a few simple patterns

Read variables, functions and events; update webpage text; and build one helpful service-information button.

KEY IDEA Aim for useful interaction, not advanced software programming.

A webpage can respond to a visitor

Static information

A public service page can display opening hours, contact details and instructions without any scripting. This may be all visitors need.

Interactive information

A visitor may select a service and reveal more guidance, or receive feedback after a form check. JavaScript can make the response immediate.

KEY IDEA Use scripting when it helps a real visitor complete a task.

HTML, CSS and JavaScript: three roles

HTML — the content

HTML supplies headings, links, forms, buttons and the meaning of the information. It answers: What is on the page?

CSS and JavaScript

CSS sets colours, spacing and responsive layout. JavaScript responds to events and updates elements. They answer: How does it look? What does it do?

KEY IDEA A well-designed webpage may use all three, but not every page needs complex scripts.

Example: a library opening-hours button

HTML

A button labelled “Show opening hours” and an empty paragraph are defined in HTML. The button is a real control, not decorative text.

JavaScript

A click event causes the paragraph to show a prepared message such as “Mon–Fri 08:00–17:00”. No page reload is needed.

KEY IDEA One small interaction can make useful public information easier to find.

Where browser JavaScript runs

Browser-side code

A client-side script runs in the visitor’s browser. It may examine an element, respond to a click or change visible text.

Server-side work

A server may retrieve catalogue records, authenticate staff and enforce permissions. The browser’s JavaScript cannot replace those protected operations.

KEY IDEA An attractive interactive interface is not the same as a secure records service.

When is JavaScript useful?

Useful tasks

Reveal general guidance, filter a short list already present on a page, count practice clicks or check whether a required field was filled in.

Unhelpful uses

Unnecessary alerts, moving controls, flashing effects or hidden content can make a page harder to use, especially for keyboard and screen-reader users.

KEY IDEA Choose interactions according to user needs rather than decoration.

Adding JavaScript inside an HTML page

Internal script

An HTML script element contains JavaScript instructions. A script placed near the bottom of the body usually runs after the elements above it exist.

Example

The instruction console.log("Ready"); writes a message to the developer console. It does not display Ready as part of the webpage.

KEY IDEA A script element introduces JavaScript; a browser then executes its instructions.

Keeping JavaScript in a separate file

External script

An HTML page may include <script src="script.js" defer></script>. The src path identifies a separate JavaScript file.

Why separate files?

External scripts help organise code and can be reused across several webpages. For traditional scripts, defer waits until HTML parsing has finished.

KEY IDEA For the practical, use index.html for structure and script.js for behaviour.

What is a JavaScript statement?

Instruction

A statement tells the browser to perform an action or define information. JavaScript is case-sensitive: message and Message are different names.

Comments

A single-line comment begins with // and helps a reader understand code. It is not executed as a JavaScript instruction.

KEY IDEA Read each statement in terms of the action it is intended to perform.

Strings, numbers and Boolean values

Text and numbers

"Library" is a string because it is quoted. 6 is a number, which may be used in a simple count.

True or false

true and false are Boolean values. They express simple conditions such as whether a section should be expanded.

KEY IDEA Data types help the script know whether it is working with text, numbers or conditions.

Variables: const and let

const

The statement const service = "Library"; creates a named binding that cannot be reassigned. It is suitable for a reference that stays the same.

let

The statement let clicks = 0; creates a variable that can be reassigned. Each click may update it to clicks = clicks + 1;.

KEY IDEA Use meaningful names so the purpose of the code is easy to follow.

Worked example: a practice counter

Starting value

let visitors = 0; defines a counter. It does not itself draw anything on the page.

Updating the value

When a button is clicked, visitors = visitors + 1; increases the number. The script can then update a paragraph to show the new count.

KEY IDEA Changing a variable and changing webpage text are two separate steps.

Functions package useful instructions

What is a function?

A function groups instructions that can run when needed. Naming a function gives the reader a useful description of its purpose.

Example

function showMessage() { return "Ask at the desk"; } defines a function. Calling showMessage() returns the text.

KEY IDEA A function is a reusable set of instructions, not an HTML element.

Conditions: making a simple decision

if statement

An if statement runs its block only when a condition is true. Conditions can help decide which explanatory message to show.

Example

If the number of entered characters is less than three, the page can ask for a longer example reference; otherwise it can show a practice confirmation.

KEY IDEA Conditional feedback should be accurate, specific and easy to understand.

The Document Object Model (DOM)

A representation of HTML

After parsing HTML, the browser provides a structured representation of elements called the DOM. JavaScript can find these elements.

Why it matters

A script needs to know exactly which button or paragraph to interact with. IDs and other selectors provide a way to locate them.

KEY IDEA The DOM connects JavaScript instructions to HTML elements.

Identify an element with an HTML id

HTML

Welcome. gives the paragraph a unique ID. The visible word is its current text.

JavaScript

document.getElementById("notice") retrieves the element with that ID. Spelling and capitalisation must match.

KEY IDEA A wrong ID is one of the most common beginner scripting errors.

Update text using textContent

JavaScript

notice.textContent = "Opening hours: 08:00–17:00"; replaces text inside the chosen paragraph.

Why textContent?

For ordinary messages it inserts text rather than interpreting the string as HTML. It is an appropriate choice for simple service feedback.

KEY IDEA Changing text on the page does not imply that any database was updated.

Events: actions that trigger code

Common events

A click event happens when a visitor activates a button. Input and change events may occur when a form control is edited.

A browser response

A listener waits for a specific event. When that event occurs, the linked function runs.

KEY IDEA An event is something that happens; a listener specifies the response.

addEventListener connects action and code

Pattern

button.addEventListener("click", function () { ... }); attaches a response to a button click.

What happens next?

When visitors activate the button, the function inside the listener runs and may update a message element.

KEY IDEA A clear button label tells the visitor what will happen before they click.

A complete beginner button example

HTML code

```
<button id="showInfo">Show
information</button>
<p id="message">Select the button.</p>
```

JS code

```
document.getElementById("showInfo")
 .addEventListener("click", () => {
  document.getElementById("message")
   .textContent = "Ask at the service
desk.";
 });
```

KEY IDEA The two IDs connect the script to the correct HTML elements.

Follow the click step by step

Before the click

The browser displays the button and the message “Select the button”. The listener is ready, but has not run.

After the click

The browser runs the listener function and replaces the paragraph with the public-service instruction.

KEY IDEA An event can change part of a page without reloading the entire document.

Use a real button for an action

Keyboard access

A native <button> is usually focusable and responds to keyboard activation. It also exposes an appropriate role to assistive technology.

Visible feedback

The result should be clear. An aria-live="polite" region may announce changed status text to a screen reader.

KEY IDEA Interactive design must be usable beyond mouse clicks.

Forms and immediate feedback

Helpful check

A browser can tell a visitor that a required field is empty or that a practice reference is too short.

Keep the message specific

Instead of “Invalid”, write “Enter at least three characters in the service reference field”. Do not communicate the error using colour alone.

KEY IDEA Useful feedback helps a visitor understand how to correct the problem.

A basic submit event

Event handling

A script can listen for a form’s submit event. The handler can display a practice message on the page.

preventDefault()

This method prevents the usual browser submission action. In a lesson demonstration, it can keep the example on the current page.

KEY IDEA Preventing submission is not the same as sending or saving information.

HTML validation and JavaScript checks

Start with HTML

Use appropriate input types, visible labels and required attributes. These support basic input checks in the browser.

Add JavaScript selectively

JavaScript can improve wording or show contextual help; secure applications still need trusted validation on the server.

KEY IDEA A convenient interface does not guarantee valid or secure data.

JavaScript does not replace a backend

What JavaScript can do here

Respond to a click, alter visible text, check simple inputs and manage the current page’s interface.

What it cannot guarantee

It cannot securely grant access to confidential records, authenticate staff or reliably store institutional requests without a trusted system behind it.

KEY IDEA Never rely on client-side code alone to protect sensitive information.

Privacy in library and records examples

Use fictional content

Practise with made-up names, request codes and general service information. Avoid showing real borrowers or restricted records.

Protect production services

Real institutions must consider permission checks, secure data transfer, audit requirements and records retention policies.

KEY IDEA Practice examples should not expose personally identifiable or confidential information.

Library applications

Service guidance

A small button can reveal loan rules, opening hours or which desk provides a service.

Simple browsing aid

A script may filter a short fictional list already shown on the webpage. A real catalogue normally requires a suitable back-end catalogue service.

KEY IDEA JavaScript improves the presentation of public information; it does not replace a library system.

Records-management applications

Public guidance

Selecting an enquiry category may reveal which office handles a public request and what general procedure to follow.

Request form feedback

A practice page may report missing required fields before the visitor submits a real request to an approved service.

KEY IDEA The script must not expose, retrieve or alter restricted records without secure authorisation.

Common beginner problems

Wrong or missing ID

If getElementById("notice") does not match the HTML id, the script may fail to find the element.

Wrong timing or spelling

Running code before HTML exists, using the wrong filename, or mistyping JavaScript names can cause errors. Check the console when troubleshooting.

KEY IDEA When a script fails, verify element IDs, file paths and browser console messages.

Practical: make an information button

First, create the HTML

Include a heading, a button with an ID and a paragraph with another ID.

Then, add script.js

Locate the two elements, attach a click listener and update the paragraph with a fictional library or records service instruction.

KEY IDEA A successful page updates the message when the button is activated.

What makes this practical successful?

Expected evidence

The browser shows the heading and button; after a click it reveals a short, clear service message.

Quality checks

The button works with a keyboard, the script file loads correctly and the message contains no real personal or confidential data.

KEY IDEA Small working examples are more valuable than complicated code copied without understanding.

Assignment 10: one useful button

What to build

Create one simple webpage for a fictional library or records centre. Include a heading, an action button and JavaScript that changes one paragraph.

What to prepare

Submit index.html, script.js and one screenshot showing the message after the button was activated. No server or database is required.

KEY IDEA Keep the webpage focused on one useful interaction.

Check your understanding

Knowledge check

Explain the different roles of HTML, CSS and JavaScript, what a DOM element is, and how an event listener works.

Practical check

Use the quiz to read corrections for incorrect answers. Repeat the quiz and change a message in the live JavaScript editor.

KEY IDEA Understanding one simple interaction prepares you to learn about larger information systems.

Unit 10: key takeaways

What you should remember

JavaScript adds behaviour to an HTML/CSS page. Events can trigger functions that locate DOM elements and update text.

Where it fits professionally

Libraries and records centres can use simple interactions for public guidance and form feedback, while secure services rely on trusted back-end systems.

KEY IDEA Next: Explore web technologies used in information institutions.